

BEGINNER PROJECT HANDBOOK

CareTrack AI (THRIVE)

Complete Project Walkthrough & Demo Handbook

Table of Contents

1. Complete Project Walkthrough & Demo Handbook
2. Contents
3. What is CareTrack AI?
4. What problem does it solve?
5. Who uses it?
6. Why is it different?
7. What happens when a patient is high risk?
8. 30-second explanation
9. 1-minute explanation
10. 3-minute explanation
11. 5-minute technical explanation
12. Outpatient follow-up
13. Patient friction
14. Why prioritization matters
15. Why explainability matters
16. Separate facts from pitch claims
17. Predict
18. Explain
19. Rank
20. Intervene
21. Simulate and follow up
22. Complete example: Priya Patel
23. High-level diagram

24. Current deployment architecture
25. What happens when the browser opens the site?
26. What happens when a user logs in?
27. What happens when the dashboard loads?
28. What happens when a patient is opened?
29. What happens when risk is calculated?
30. What happens when the simulator is used?
31. What happens when a patient is contacted?
32. What happens when Gemini is asked a question?
33. What happens when CSV export is used?
34. Root files
35. src/
36. src/context/
37. src/pages/
38. src/components/
39. src/services/
40. src/db/
41. src/data/
42. Prerequisites
43. Install dependencies
44. Create local environment configuration
45. Development command
46. Production-style local build
47. TypeScript check
48. Local Docker option
49. First successful local run checklist
50. Public versus secret variables
51. Render environment behavior
52. Production database requirement
53. Login flow
54. Logout flow
55. Roles in the frontend
56. Roles in the backend
57. Seed accounts

58. [Login screen](#)
59. [Main layout](#)
60. [Executive Dashboard](#)
61. [Priority Risk Queue](#)
62. [Patient Records & Directory](#)
63. [Patient Details](#)
64. [Clinical Risk Analyzer](#)
65. [Outreach and Dispatch](#)
66. [Analytics](#)
67. [Settings](#)
68. [Global modals and drawer](#)
69. [Default weights](#)
70. [Actual formulas](#)
71. [Total and tier](#)
72. [Scoring output](#)
73. [Important interpretation rule](#)
74. [Creating a patient](#)
75. [Reading a patient list](#)
76. [Updating a patient](#)
77. [Appointment history](#)
78. [Intervention history](#)
79. [Notification history](#)
80. [Public routes](#)
81. [Authenticated routes](#)
82. [Admin-protected route](#)
83. [API request example](#)
84. [Connection order](#)
85. [Tables](#)
86. [Startup and seeding](#)
87. [Persistence warning](#)
88. [Embedded PostgreSQL warning](#)
89. [Where it lives](#)
90. [What Gemini receives](#)
91. [Example questions](#)

- 92. Fallback mode
- 93. What is not implemented
- 94. Demo steps
- 95. Map component
- 96. How to access it
- 97. How distance is calculated
- 98. Presentation-guide discrepancy
- 99. Contact channels
- 100. Actual provider behavior
- 101. What is recorded?
- 102. Demo steps
- 103. Audit events
- 104. Audit screen/access
- 105. What is not logged?
- 106. CSV export
- 107. Claims to avoid
- 108. 0:00-0:30: Login
- 109. 0:30-1:00: Dashboard
- 110. 1:00-1:35: Risk Queue
- 111. 1:35-2:00: Priya Patel details
- 112. 2:00-2:20: What-If simulator
- 113. 2:20-2:40: Contact workflow
- 114. 2:40-3:00: Gemini and audit
- 115. Slide 1: Title and vision
- 116. Slide 2: Problem
- 117. Slide 3: Solution
- 118. Slide 4: Scoring engine
- 119. Slide 5: Hero patient
- 120. Slide 6: GIS
- 121. Slide 7: Gemini
- 122. Slide 8: Architecture
- 123. Slide 9: Roles and governance
- 124. Slide 10: Impact
- 125. Slide 11: Differentiation

126. Slide 12: Roadmap
127. Render Blueprint
128. Deployment steps
129. Expected production startup
130. Production checks
131. Docker deployment
132. Before changing anything
133. Common change locations
134. Do not make these assumptions
135. Safe feature change pattern
136. npm start says package.json is missing
137. Port 3000 is already in use
138. Embedded PostgreSQL startup is slow
139. .pgdata is not empty / database restarts strangely
140. Production says DATABASE_URL must be configured
141. PostgreSQL connection fails
142. Login fails
143. Dashboard is blank or shows fallback values
144. Gemini assistant shows fallback text
145. Twilio says demo mode
146. Map search does not return results
147. TypeScript errors after a change
148. Render health check fails
149. Why deterministic scoring instead of deep learning?
150. Does the score predict readmission?
151. What happens if PostgreSQL is unavailable?
152. Is the data real?
153. Is the system HIPAA or DPDPA compliant?
154. Is Gemini making the risk decision?
155. What if Gemini is unavailable?
156. Is telehealth integrated?
157. Is Twilio fully integrated?
158. Is the map a routing system?
159. What does Render deploy?

160. What is the strongest demo point?

161. What is still roadmap?

162. Before a demo

163. Before changing code

164. Before deploying Render

165. One-sentence project summary

CareTrack AI (THRIVE)

Complete Project Walkthrough & Demo Handbook

This handbook explains the CareTrack AI project as it exists in the repository. It is written for a teammate who is new to the codebase and may be new to software development.

The code is the source of truth. The supplied presentation guide is useful for the story and demo order, but some presentation claims are aspirational or outdated. Those differences are called out explicitly in this document.

Current deployment truth: the project is a Vite + React frontend served by an Express/Node server and deployed as a Render web service with PostgreSQL. It is not currently a Vercel serverless project.

Contents

1. [Executive Summary](#)
2. [The Problem](#)
3. [The Core Idea](#)
4. [Complete System Architecture](#)
5. [Project Folder Structure](#)
6. [How to Run Locally](#)
7. [Environment Variables](#)
8. [Authentication and Roles](#)
9. [Frontend Walkthrough](#)
10. [Risk Scoring Engine](#)
11. [Patient Data Flow](#)
12. [Backend API Reference](#)

13. [Database Architecture](#)
 14. [Gemini Assistant](#)
 15. [GIS and Map Features](#)
 16. [Communication and Twilio](#)
 17. [Audit Logging and CSV Export](#)
 18. [Presentation Claims Compared with Code](#)
 19. [Three-Minute Demo Script](#)
 20. [Longer Presentation Scripts](#)
 21. [Production Deployment](#)
 22. [Safe Modification Guide](#)
 23. [Troubleshooting](#)
 24. [Judge Q&A](#)
 25. [Final Beginner Checklist](#)
-

1. Executive Summary

What is CareTrack AI?

CareTrack AI is a hospital operations application for identifying outpatients who may miss important follow-up appointments.

It combines:

- A React dashboard for hospital staff.
- An Express API for authentication, patient records, risk calculations, interventions, and analytics.
- A deterministic scoring engine that converts patient and appointment facts into a 0-100 follow-up risk score.
- PostgreSQL persistence for staff, patients, appointments, predictions, interventions, notifications, audit records, and scoring configuration.
- Optional Gemini assistance for operational questions.
- Optional live Twilio SMS delivery, with a built-in demo mode when Twilio is not configured.

- A Leaflet/OpenStreetMap map picker for patient address and distance entry.

The product is a decision-support and coordination tool. It does not diagnose illness, prescribe medicine, or replace a clinician.

What problem does it solve?

A normal reminder system treats every patient the same. CareTrack AI attempts to answer four operational questions:

1. Which patients are most likely to miss follow-up?
2. Why does the system think they are at risk?
3. Which patients should staff contact first?
4. What action can staff record or simulate to reduce the practical barrier?

The application focuses on barriers represented in the project data:

- Missed appointment history.
- Distance from the hospital.
- Historical attendance rate.
- Long appointment intervals.
- Long treatment duration.
- Age-related mobility or dependency signals.

Who uses it?

The seed data includes five staff roles:

ROLE	EXAMPLE ACCOUNT	INTENDED USE IN THE UI
DOCTOR	doctor@caretrack.in	Review risk, patients, and clinical follow-up context
NURSE	nurse@caretrack.in	Work the outreach and intervention queue

ROLE	EXAMPLE ACCOUNT	INTENDED USE IN THE UI
COORDINATOR	coordinator@caretrack.in	Coordinate follow-up contact, transport, and teleconsultation offers
CARE_MANAGER	caremanager@caretrack.in	Review clinic-wide trends and adherence operations
ADMIN	admin@caretrack.in	Use settings, staff management, scoring configuration, and demo controls

The default seed password for the demo accounts is password123 . These credentials are for demonstration only and must not be used in a real hospital deployment.

Why is it different?

The intended difference is not that it predicts with a black-box machine-learning model. The current implementation uses explicit formulas and displays the contributing factors. That makes the score easy to inspect and explain during a demo.

The closed-loop concept is:

Predict -> Explain -> Rank -> Intervene -> Record outcome

The project also includes a hypothetical simulator. A coordinator can change values such as distance or missed appointments and immediately see how the score would change. The simulation itself does not change the database until a separate action is submitted.

What happens when a patient is high risk?

The backend calculates a risk tier and recommended actions. The UI can then:

1. Show the patient in the Priority Risk Queue.
2. Open the patient's details and factor explanation.
3. Open the What-If simulator.
4. Open the action recommendation modal.

5. Record an intervention.
6. Contact the patient through phone-call, SMS, or WhatsApp workflow.
7. Update the intervention status later.
8. Include the action in analytics and audit history.

Some actions are simulated. In particular, phone calls and WhatsApp messages are not connected to a live provider in the current implementation.

30-second explanation

CareTrack AI helps hospital staff find outpatient follow-up patients who may miss their next visit. It calculates an explainable 0-100 score from missed visits, travel distance, attendance, appointment cadence, treatment duration, and age. Staff can inspect the reason, prioritize the queue, simulate a solution such as teleconsultation, and record outreach. PostgreSQL stores the operational history, while Gemini is an optional assistant for operational questions.

1-minute explanation

Hospitals lose capacity when follow-up appointments are missed, but a generic reminder does not explain who needs personal help. CareTrack AI turns patient and appointment information into a transparent risk score. The dashboard ranks high-risk patients, and each score is broken into named factors such as missed appointments and travel distance. Staff can open a patient, see the appointment timeline, test a hypothetical telehealth option, send or simulate outreach, and record the result. The application is a React frontend backed by Express and PostgreSQL. Gemini can summarize operational risk questions, but the deterministic engine remains the source of the score.

3-minute explanation

The application begins with staff authentication. After login, the dashboard loads a cohort summary, top high-risk patients, risk distribution, and operational trends. The Risk Queue supports search, risk filters, intervention filters, due-date filters, sorting, pagination, and CSV export. Opening a patient loads demographics, appointments, interventions, and a current risk analysis. The scoring engine uses six bounded factors whose configured maximums add to 100. For example, the canonical Priya Patel record has 5 missed appointments, a 42 km distance, 58% attendance, a 60-day cadence, and a 12-month treatment duration. The current formula produces 83 and classifies her as CRITICAL. The What-If simulator can set distance to zero to represent teleconsultation and shows 64, a 19-point hypothetical reduction. Staff can then open the contact modal, select an outreach channel, and save the attempt. PostgreSQL stores the resulting intervention, notification, and audit records.

5-minute technical explanation

CareTrack AI is a single full-stack TypeScript application. Vite builds the React client into `dist` ; esbuild bundles `server.ts` into `dist-server/server.cjs` ; Express serves both the API and the built client in production. The frontend uses `AppContext` for in-memory UI state and `AuthContext` for the current user and token. `src/services/api.ts` maps UI actions to relative `/api` calls. Express applies JSON parsing, CORS headers, database initialization middleware, JWT authentication, and route handlers. `src/db/db.ts` first attempts `DATABASEURL` , then local PostgreSQL, then embedded PostgreSQL for local development, and finally an in-memory fallback if permitted. On Render, `render.yaml` creates the web service and PostgreSQL database and supplies `DATABASE URL` . The scoring engine is deterministic and returns factor contributions, reasons, protective factors, recommended actions, and a tier. Gemini is lazily initialized only when `GEMINIAPIKEY` exists and uses the current `gemini-3.6-flash` model through `@google/genai` . Twilio SMS is optional and falls back to a recorded demo result. The current code is suitable for a hackathon or controlled demo, but security and authorization hardening would be required before handling real protected health information.

2. The Problem We Are Solving

Outpatient follow-up

Many treatments require repeated visits after an initial appointment, procedure, or diagnosis. A missed follow-up can delay review, rehabilitation, medication checks, diagnostic testing, or care coordination.

CareTrack AI models the operational side of that problem. It does not claim to determine medical outcomes. It asks whether the patient appears difficult to keep engaged with the next scheduled follow-up.

Patient friction

The project represents friction using data fields such as:

- `missedAppointments` .
- `distanceKm` .
- `attendanceRate` .
- `appointmentFrequencyDays` .
- `treatmentDurationMonths` .
- `age` .
- `transportAccess` .
- `preferredLanguage` .

These values are used for scoring, display, or outreach context. They are not all clinical measurements.

Why prioritization matters

A clinic team cannot make a personal phone call to every patient at the same time. A ranked queue gives staff a starting point. In this application, the sort order is based on the calculated `currentRisk.score` , with filters for risk level, intervention status, and due-date windows.

Why explainability matters

A score such as **83** is not useful by itself. The application returns:

- A risk level.
- The individual factor contributions.
- Human-readable reasons.
- Protective factors.
- Recommended actions.
- An immediate action, secondary action, and alternative action.
- A natural-language summary.

This is why the product story uses “explainable” rather than simply “predictive.”

Separate facts from pitch claims

The presentation guide includes statistics such as 25%-30% no-show rates, 3x readmission risk, \$150B annual loss, ROI percentages, and recovered capacity. Those figures are not calculated or sourced by this repository. Present them only as external context if you have independently verified sources. Do not describe them as measured results from this codebase.

The current repository does not contain:

- A hospital production dataset.
- A validated clinical outcomes study.
- A financial ROI model.
- A readmission prediction model.
- A causal analysis showing that interventions reduce readmissions.

Use the phrase “operational decision support prototype” when accuracy matters.

3. The Core Idea

Predict

The scoring engine receives a patient-like object and computes six factor scores. It caps the final total between 0 and 100.

The main implementation is:

- `src/services/scoringEngine.ts`
- `calculatePatientRisk(...)`
- `DEFAULTSCORINGCONFIG`

The server calls the engine for patient detail views, prediction requests, analyzer requests, patient creation/update, seeded records, and dashboard calculations.

Explain

The result includes `topFactors`, `reasons`, `protectiveFactors`, and recommended actions. The UI renders those values in `RiskCard`, `RiskExplanationModal`, `RiskGauge`, `PatientDetailsPage`, and `AnalyzerFindingsModal`.

Rank

The Risk Queue calls:

```
GET /api/patients
```

with parameters such as:

```
riskLevel=HIGH  
interventionStatus=ALL  
dueFilter=NEXT_7_DAYS  
sortBy=riskScore  
sortOrder=desc  
page=1  
limit=15
```

The backend applies filtering, sorting, and pagination in `dbGetPatients(...)`.

Intervene

There are two related workflows:

1. **Intervention record:** create or update a clinical outreach record.
2. **Patient contact:** submit a phone/SMS/WhatsApp contact attempt and create a notification plus intervention record.

The relevant UI components are:

- `InterventionModal.tsx`.
- `ContactPatientModal.tsx`.
- `WhatShouldIDoModal.tsx`.
- `AnalyzerFindingsModal.tsx`.

Simulate and follow up

`RiskSimulator.tsx` is a client-side hypothetical calculator. It starts with the current patient values, exposes sliders, and can set effective distance to `0 km` when “Simulate Telehealth” is active. It does not call the backend while the slider moves and does not save the simulated values automatically.

Complete example: Priya Patel

The canonical seed record is:

FIELD	VALUE
Internal id	<code>PAT-1042</code>
Patient code	<code>P-1042</code>

FIELD	VALUE
Name	Priya Patel
Age	46
Condition	Post-CABG Cardiac Rehabilitation & Hypertension
Distance	42 km
Appointments	12 total, 7 attended, 5 missed
Attendance	58%
Cadence	Every 60 days
Treatment duration	12 months
Transport	Requires Assistance
Language	Gujarati

Using the default scoring configuration:

Missed visits: 35

Distance: 19

Attendance: 17

Cadence: 7

Duration: 5

Age: 0

--

Total: 83 -> CRITICAL

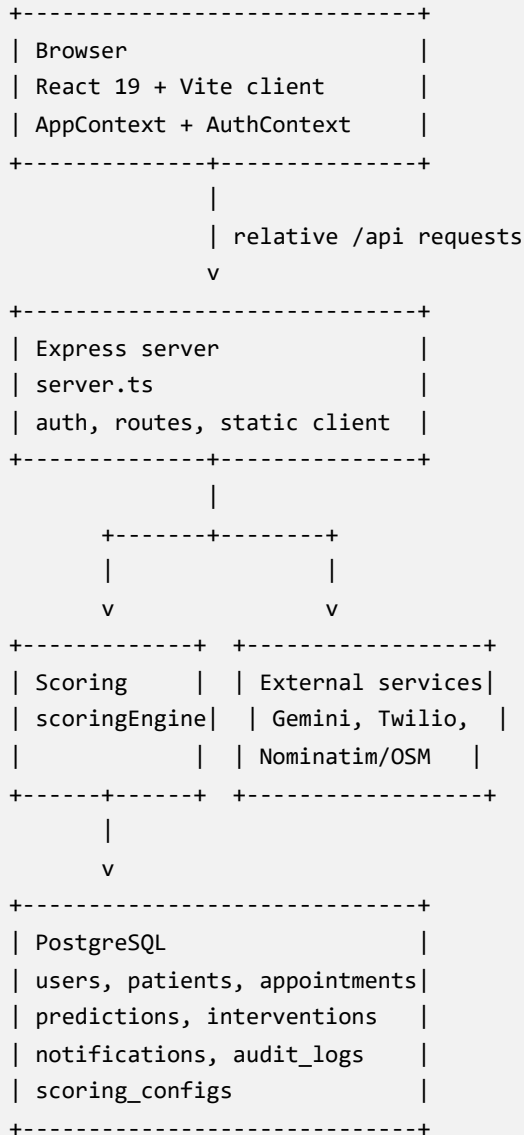
The simulator sets effective distance to zero for the telehealth scenario:

83 original - 19 distance points = 64 simulated -> HIGH

This is a hypothetical risk recalculation, not proof that a telehealth appointment will occur or that a clinical outcome will improve.

4. Complete System Architecture

High-level diagram



Current deployment architecture

Render runs one long-lived Node web service:

Render web service

- > npm ci && npm run build
- > npm start
- > node dist-server/server.cjs
- > Express listens on 0.0.0.0:\$PORT
- > Express serves dist/index.html and /api/*
- > Render PostgreSQL is exposed through DATABASE_URL

The deployment definition is `render.yaml` . It creates:

- A web service named `caretrack-ai` .
- A PostgreSQL database named `caretrack-db` .
- A generated `JWT_SECRET` .
- `NODE_ENV=production` .
- `PGMAXCONNECTIONS=5` .
- A health check at `/api/health` .

What happens when the browser opens the site?

1. The browser requests `/` .
2. Express serves `dist/index.html` in production.
3. The built JavaScript bundle loads.
4. `src/main.tsx` renders `App` .
5. `AuthProvider` checks `localStorage` for `caretrackuser` and `caretrack token` .
6. If both exist, the application restores the session locally.
7. If not, `AuthGate` shows `LoginPage` .
8. There is no React Router. The visible page is selected using `currentPage` state in `AppContext` .

What happens when a user logs in?

1. The user enters an email and password in `LoginPage` .
2. `AuthContext.login(...)` calls `api.login(...)` .
3. `src/services/api.ts` sends `POST /api/auth/login` .

4. `server.ts` normalizes the email and loads the user with `dbGetUserByEmail(...)`.
5. Seed/demo passwords and bcrypt hashes are supported.
6. The server signs a JWT using `JWT_SECRET`, or the demo fallback secret when not configured outside production.
7. The response contains `success`, `token`, and a safe user object without `passwordHash`.
8. The browser stores the token and user in `localStorage`.
9. `AuthGate` switches from `LoginPage` to `MainLayout`.
10. `AppContext` calls `refreshDashboard(...)`.

What happens when the dashboard loads?

1. `DashboardPage` asks `AppContext` for the shared summary.
2. `AppContext.refreshDashboard()` calls:
 - `GET /api/dashboard/summary`.
 - `GET /api/settings/config`.
1. `DashboardPage` separately calls:
 - `GET /api/patients?riskLevel=HIGH&limit=5&sortBy=riskScore&sortOrder=desc`.
 - `GET /api/dashboard/trends`.
1. The backend initializes the database once for the process.
2. PostgreSQL queries or the permitted local fallback store return data.
3. React renders KPI cards, top-risk patients, charts, and navigation actions.

Some dashboard fallback numbers exist in the UI if data is not available. They are presentation defaults, not authoritative database measurements.

What happens when a patient is opened?

1. A page calls `viewPatientDetails(patientId)`.
2. `AppContext` stores the selected internal patient id and changes `currentPage` to `patient-details`.
3. `PatientDetailsPage` calls `GET /api/patients/:id`.

4. The backend loads the patient, appointments, interventions, and scoring configuration.
5. It recalculates current risk with `calculatePatientRisk(...)`.
6. The response contains `patient`, `appointments`, `interventions`, and `riskAnalysis`.
7. The UI displays the patient profile, risk gauge, factor details, timeline, actions, simulator, and report controls.

What happens when risk is calculated?

There are three important server paths:

- `POST /api/predictions/predict` : direct prediction input.
- `GET /api/patients/:id/risk` : risk for a stored patient.
- `POST /api/analyzer/process` : analyzer response with findings and a recommended intervention.

The prediction calculation itself is deterministic. The prediction id contains randomness, but the score for identical inputs and configuration is stable.

What happens when the simulator is used?

1. `PatientDetailsPage` renders `RiskSimulator`.
2. The component copies patient values into React state.
3. Slider changes recalculate locally through `calculatePatientRisk(...)` imported from `scoringEngine.ts`.
4. The score difference is displayed beside the original score.
5. “Simulate Telehealth” sets effective distance to zero.
6. No database write occurs until the user starts a separate real workflow.

What happens when a patient is contacted?

1. The user opens `ContactPatientModal` from the dashboard, queue, directory, or patient detail view.
2. The user chooses `PHONE_CALL`, `SMS`, or `WHATSAPP` and enters/accepts contact details.

3. `api.contactPatient(...)` sends `POST /api/patients/:id/contact`.
4. The backend loads the patient and builds a message draft or uses the submitted content.
5. `sendNotification(...)` chooses live SMS or demo behavior.
6. `dbCreateNotification(...)` stores the notification record.
7. `dbCreateIntervention(...)` stores the associated intervention.
8. `dbLogAudit(...)` records the operational action.
9. The UI displays a toast and refreshes relevant data.

What happens when Gemini is asked a question?

1. `AiAssistantDrawer` displays a chat drawer.
2. The user sends a message.
3. `api.askAssistant(...)` posts to `/api/assistant/chat`.
4. The backend loads dashboard and patient context.
5. If `GEMINI_API_KEY` is present, `getGeminiClient()` creates a lazy `GoogleGenAI` client.
6. `ai.models.generateContent(...)` is called with model `gemini-3.6-flash`.
7. The prompt includes strict operational-only instructions.
8. If Gemini is unavailable or errors, the server creates a deterministic keyword-based fallback response.
9. The UI displays the returned reply.

What happens when CSV export is used?

1. `RiskQueuePage` calls `api.exportCsv()`.
2. The browser requests `GET /api/export/csv` with the JWT.
3. The backend loads up to 1,000 patients, sorted by risk.
4. It constructs CSV headers and rows.
5. It responds with `text/csv` and a download filename.
6. The browser creates a Blob download.

The exporter quotes selected text fields but is not a full RFC-compliant CSV escaping library. Treat it as a demo/export utility, not a compliance-grade export without further hardening.

5. Project Folder Structure

The relevant repository tree is:

```
NextEra---ASA-Trinity/
├── .env.example
├── .gitignore
├── Dockerfile
├── README.md
├── docker-compose.yml
├── index.html
├── metadata.json
├── package.json
├── package-lock.json
├── render.yaml
├── server.ts
├── tsconfig.json
├── vite.config.ts
├── src/
│   ├── App.tsx
│   ├── index.css
│   ├── main.tsx
│   ├── types.ts
│   ├── components/
│   ├── context/
│   ├── data/
│   ├── db/
│   ├── pages/
│   └── services/
└── docs/
    └── CARETRACK_AI_COMPLETE_WALKTHROUGH.md
```

The old `api/` directory may remain as an empty local directory after the Vercel migration. It is not part of the current active deployment path. There are no active API wrapper files under it.

Root files

`package.json`

Defines project metadata, scripts, runtime dependencies, and development dependencies.

Important scripts:

SCRIPT	MEANING
<code>npm run dev</code>	Runs <code>tsx server.ts</code> for local development
<code>npm run build:client</code>	Builds the React/Vite client into <code>dist</code>
<code>npm run build:server</code>	Bundles <code>server.ts</code> into <code>dist-server/server.cjs</code>
<code>npm run build</code>	Runs the client and server builds
<code>npm start</code>	Runs the compiled production server
<code>npm run clean</code>	Deletes <code>dist</code> and <code>dist-server</code>
<code>npm run lint</code>	Runs <code>tsc --noEmit</code> ; this is a TypeScript check, not ESLint

Do not casually remove dependencies from this file. The frontend and backend share one dependency graph.

package-lock.json

Locks npm dependency versions. Use `npm ci` in deployment and clean CI environments.

server.ts

This is the main backend entry point and the production server entry point. It contains:

- Express app creation.
- JSON body parsing.
- CORS headers.
- Database initialization middleware.
- JWT authentication middleware.
- Role-checking middleware.
- All REST endpoints.
- Gemini integration.
- Static client serving.
- Vite middleware for local development.

- The `startServer()` function.

This is the most sensitive file in the project. A route change can affect authentication, database writes, UI workflows, and production deployment.

`render.yaml`

Defines the current Render Blueprint:

- Node web service.
- `npm ci && npm run build` build command.
- `npm start` start command.
- `/api/health` health check.
- Managed PostgreSQL database.
- Render-generated `JWT_SECRET`.

`Dockerfile`

Builds a Node 22 Alpine production image. It builds both the client and server and starts `dist-server/server.cjs`.

`docker-compose.yml`

Runs a local Docker PostgreSQL 16 container and a production-style app container. It is useful for local/container testing and is separate from Render deployment.

`vite.config.ts`

Configures Vite, React, Tailwind, and the `@` path alias. The alias maps to the repository root.

`tsconfig.json`

Configures TypeScript with:

- ES2022 target.
- DOM libraries.
- Bundler module resolution.
- React JSX transform.

- `@/*` path aliases.
- No emitted TypeScript files.

`index.html`

The Vite HTML entry document. It contains the root element and loads `/src/main.tsx` during development/build processing.

`.env.example`

Lists environment variable names only. It must not contain real secrets.

`.gitignore`

Ignores `node_modules`, build outputs, `.env` files, `.pgdata`, coverage, and similar generated artifacts.

`src/`

`src/main.tsx`

Creates the React root and renders `<App />`.

`src/App.tsx`

Creates the provider tree and authenticated layout:

```
AuthProvider
  -> AppProvider
    -> AuthGate
      -> LoginPage OR MainLayout
```

`MainLayout` mounts the navbar, sidebar, active page, global modals, AI drawer, and toast container.

`src/types.ts`

Contains shared TypeScript contracts, including:

- `Patient` .
- `StaffUser` .
- `Appointment` .
- `RiskPrediction` .
- `RiskFactorContribution` .
- `Intervention` .
- `AuditLog` .
- `NotificationRecord` .
- `ScoringConfiguration` .
- `AnalyzerFindings` .

If you change a shared type, search all references before saving the change.

`src/context/`

`AuthContext.tsx`

Owns frontend authentication state:

- Current user.
- Authentication status.
- Login and admin quick login.
- Registration request.
- Logout.
- Local storage token/user restoration.

`AppContext.tsx`

Owns shared application UI state:

- Current page.
- Selected patient.
- Modal visibility and selected modal patient.
- AI drawer state.

- Dashboard summary.
- Scoring configuration.
- Toasts.
- Refresh triggers.
- Demo reset action.

This is state navigation, not URL routing. Refreshing the browser does not preserve the selected page in the URL.

src/pages/

FILE	VISIBLE PAGE	MAIN PURPOSE
LoginPage.tsx	Login	Authenticate staff and offer quick demo accounts
DashboardPage.tsx	Executive Dashboard	KPIs, charts, top high-risk patients, shortcuts
RiskQueuePage.tsx	Priority Risk Queue	Search, filters, ranking, actions, CSV export
PatientDetailsPage.tsx	Patient Details	Profile, risk, appointments, interventions, simulator
PatientsPage.tsx	Patient Records & Directory	Search, filter, enrollment, map picker, admin workflows
PredictionPage.tsx	Clinical Risk Analyzer	Adjust inputs and inspect analyzer result
InterventionsPage.tsx	Patient Outreach & Dispatch	Review and update intervention records
AnalyticsPage.tsx	Follow-up Impact & Analytics	Trends, charts, and presentation-oriented metrics
SettingsPage.tsx	Scoring Model Config	Scoring controls and staff management UI
ApiDocsPage.tsx	Not currently linked	Static API documentation page component

FILE	VISIBLE PAGE	MAIN PURPOSE
RegisterPage.tsx	Not currently linked	Registration page component; API registration is still available

src/components/

FILE	PURPOSE
Navbar.tsx	Brand/header, dashboard shortcut, risk alert, AI drawer, logout
Sidebar.tsx	State-based navigation and admin-only Settings visibility
RiskCard.tsx	Compact risk card and action buttons
RiskGauge.tsx	Visual score gauge
RiskExplanationModal.tsx	Factor breakdown, reasons, protective factors, actions
WhatShouldIDoModal.tsx	Recommended intervention decision modal
InterventionModal.tsx	Manually create an intervention record
ContactPatientModal.tsx	Choose channel and record contact/outreach
AnalyzerFindingsModal.tsx	Display analyzer findings and continue to contact flow
EditPatientModal.tsx	Edit patient fields and refresh data
PdfReportModal.tsx	Browser print / Save as PDF workflow; no server PDF engine
AiAssistantDrawer.tsx	Gemini/fallback operations chat
RiskSimulator.tsx	Client-side hypothetical score changes
AppointmentTimeline.tsx	Appointment history display

FILE	PURPOSE
MapAddressPicker.tsx	Leaflet map, geocoding, GPS, distance calculation
Toast.tsx	Global temporary success/info/warning/error messages
AdminLoginModal.tsx	Admin authentication modal used by some privileged flows
JudgeDemoBanner.tsx	Present in source but not mounted by the current App.tsx
JavaArchitectureModal.tsx	Present demonstration code for a Java architecture, not active backend code

src/services/

api.ts

The frontend API client. It uses VITEAPIBASE_URL when present, otherwise /api . It attaches:

- Authorization: Bearer <JWT> .
- x-staff-name .
- x-staff-role .
- Cache-control headers.

scoringEngine.ts

The shared deterministic scoring implementation. It is imported by backend code and the client-side simulator.

notificationService.ts

Implements Twilio SMS and demo notification behavior.

`src/db/``db.ts`

Database connection, schema creation, seed logic, PostgreSQL repository methods, and local fallback stores.

`src/data/``seedData.ts`

Contains:

- Staff seed users.
- Indian cities/localities.
- Canonical patients, including Priya Patel.
- Synthetic dataset generation.
- Seed appointments, interventions, and audit logs.

6. How to Run the Project Locally

Prerequisites

Install:

- Node.js 18 or newer. Node 22 is the tested development version.
- npm.
- Optional: Docker Desktop if you want PostgreSQL through `docker-compose`.
- Optional: a PostgreSQL installation if you do not want the embedded fallback.

The repository currently uses `package-lock.json`, so use npm for the normal workflow.

Install dependencies

From the repository root:

```
npm install
```

This creates `node_modules` and installs the dependencies in `package-lock.json`.

For a clean CI/deployment install:

```
npm ci
```

`npm ci` requires the lockfile and removes/recreates dependencies consistently.

Create local environment configuration

Copy `.env.example` to `.env`.

PowerShell:

```
Copy-Item .env.example .env
```

At minimum for a local demo, you may use:

```
NODE_ENV=development
PORT=3000
JWT_SECRET=local-only-demo-secret
PG_MAX_CONNECTIONS=5
```

If `DATABASE_URL` is omitted during development, the database layer tries local PostgreSQL and then embedded PostgreSQL under `.pgdata`. The embedded PostgreSQL package may be slow on first startup because it initializes a local cluster.

For a persistent local PostgreSQL connection, set:


```
DATABASE_URL=postgresql://USER:PASSWORD@HOST:5432/caretrack
```

For Gemini:

```
GEMINI_API_KEY=your-key-for-local-testing
```

For optional live SMS:

```
TWILIO_ACCOUNT_SID=...  
TWILIO_AUTH_TOKEN=...  
TWILIO_PHONE_NUMBER=...
```

Never commit `.env` or real secrets.

Development command

```
npm run dev
```

This runs `tsx server.ts`. The server starts Express and mounts Vite middleware when a production build is not being served. Open:

```
http://localhost:3000
```

Production-style local build

```
npm run build  
npm start
```

`npm run build` does two jobs:

1. `vite build` creates the browser assets in `dist`.
2. `npm run build:server` creates `dist-server/server.cjs`.

`npm start` runs the compiled server. It serves the built client and the API from one process.

TypeScript check

```
npm run lint
```

Despite the script name, this runs `tsc --noEmit`. It catches TypeScript errors but does not run ESLint rules.

Local Docker option

```
docker compose up --build
```

This starts:

- A PostgreSQL 16 container named `caretrack-postgres`.
- An app container on port `3000`.

The Compose database and the Render database are separate environments. Do not copy the Compose password into production.

First successful local run checklist

1. The server prints that it is listening on port 3000.
2. Visit `http://localhost:3000`.
3. Log in with `admin@caretrack.in` / `password123`.
4. Confirm the dashboard loads.
5. Search for `P-1042`.
6. Open the patient and inspect the risk score.
7. Open the AI drawer. Without `GEMINIAPIKEY`, the deterministic fallback should still answer supported questions.

7. Environment Variables

VARIABLE	REQUIRED?	USED BY	MEANING
DATABASE_URL	Required in production	src/db/db.ts , Render	PostgreSQL connection string
PGMAXCONNECTIONS	Optional	src/db/db.ts	PostgreSQL pool maximum; default is 5
JWT_SECRET	Required for secure production	server.ts , Render	Signs and verifies login tokens
NODE_ENV	Recommended	server/db startup	production enables production behavior
PORT	Render supplies it	server.ts	HTTP listening port; local default 3000
GEMINIAPIKEY	Optional	server.ts	Enables Gemini Assistant calls
TWILIOACCOUNTSID	Optional	notificationService.ts	Twilio account id
TWILIOAUTHTOKEN	Optional	notificationService.ts	Twilio credential
TWILIOPHONENUMBER	Optional	notificationService.ts	Twilio sender number
VITEAPIBASE_URL	Optional	src/services/api.ts	API base URL; default /api

Public versus secret variables

VITE variables are available to browser code after Vite builds them. Never put a secret in a VITE variable.

`DATABASEURL` , `JWT_SECRET` , `GEMINI_APIKEY` , and Twilio credentials are server-side secrets. They are read by Node code and must only be configured in the Render service environment or local `.env` .

Render environment behavior

`render.yaml` automatically supplies:

- `DATABASE_URL` from the linked database.
- A generated `JWT_SECRET` .
- `NODE_ENV=production` .
- `PGMAXCONNECTIONS=5` .

You can add optional Gemini and Twilio variables in Render's service settings.

Production database requirement

When `NODEENV=production` and `DATABASE_URL` is missing, the database initialization throws an explicit configuration error. This is intentional: production should not silently start with a temporary local database.

If a configured production `DATABASE_URL` cannot connect, inspect the connection logs. The database layer still contains local fallback branches for the existing development behavior, so verify the actual environment and connection string rather than assuming persistence.

8. Authentication and Roles

Login flow

The UI is in `src/pages/LoginPage.tsx` . The state and storage logic is in `src/context/AuthContext.tsx` .

The login request is:

```
POST /api/auth/login
Content-Type: application/json

{
  "email": "admin@caretrack.in",
  "password": "password123"
}
```

The server:

1. Trims and lowercases the email.
2. Looks up the user.
3. Supports demo aliases such as email text containing `admin` , `doctor` , `nurse` , `coordinator` , or `manager` .
4. Supports demo plaintext/placeholder passwords and bcrypt hashes.
5. Logs the login to `audit_logs` .
6. Removes `passwordHash` from the returned user object.
7. Signs a JWT containing id, role, email, and name.

The browser stores:

```
caretrack_token
caretrack_user
```

in `localStorage` .

Logout flow

Logout is frontend-driven:

1. A short visual delay is shown.
2. `caretrack_user` is removed.
3. `caretrack_token` is removed.
4. The application returns to `LoginPage` .

There is no server-side token revocation endpoint. JWT expiration is the main token lifetime control.

Roles in the frontend

`AuthContext` exposes booleans:

- `isAdmin` .
- `isDoctor` .
- `isNurse` .
- `isCoordinator` .
- `isCareManager` .

`Sidebar` only displays Settings to `ADMIN` . `PatientsPage` labels enrollment and editing as admin workflows. The UI is helpful guidance, not the complete security boundary.

Roles in the backend

`requireAuth` validates the JWT. `requireRole(...)` exists and is used for the scoring configuration update route.

The current backend role enforcement is incomplete. Several routes use `requireAuth` without a role restriction, including patient writes, user management, demo reset/generation, analyzer, and outreach. Do not claim that every action is server-side RBAC-enforced.

This is an important production hardening task, but it is outside the scope of this walkthrough document and should be planned as a separate security change.

Seed accounts

EMAIL	PASSWORD	ROLE
<code>doctor@caretrack.in</code>	<code>password123</code>	<code>DOCTOR</code>
<code>nurse@caretrack.in</code>	<code>password123</code>	<code>NURSE</code>
<code>coordinator@caretrack.in</code>	<code>password123</code>	<code>COORDINATOR</code>

EMAIL	PASSWORD	ROLE
<code>caremanager@caretrack.in</code>	<code>password123</code>	<code>CARE_MANAGER</code>
<code>admin@caretrack.in</code>	<code>password123</code>	<code>ADMIN</code>

These are demo accounts, not secure operational credentials.

9. Frontend Walkthrough

Login screen

File: `src/pages/LoginPage.tsx`

What to demonstrate:

1. Show the login form.
2. Use a quick login button if present.
3. Explain that successful login stores a JWT in browser storage.
4. Mention that the user's role controls visible navigation.

The registration page component exists, but it is not currently connected into the main login navigation. The registration API is available at `POST /api/auth/register` and the admin Settings screen can register staff.

Main layout

File: `src/App.tsx`

After authentication, the page contains:

- `Navbar` at the top.
- `Sidebar` on the left.
- One state-selected page in the main area.

- Globally mounted modal components.
- AI assistant drawer.
- Toast notifications.

There are no URL routes such as `/dashboard` or `/patients`. Navigation is internal React state. A browser refresh returns to the default page after session restoration.

Executive Dashboard

File: `src/pages/DashboardPage.tsx`

Visible areas:

- Patient Follow-up Risk Dashboard heading.
- High-risk patient count.
- Follow-ups due within seven days.
- Interventions logged and pending.
- Outreach success rate.
- Top priority high-risk follow-ups table.
- Risk distribution chart.
- Trend charts.
- Shortcuts to risk queue, interventions, analytics, patient details, risk explanation, contact, and AI assistant.

Important accuracy note: dashboard values can be loaded from the backend, but the component contains fallback display values if data is unavailable. Treat rendered fallback numbers as demo placeholders.

Priority Risk Queue

File: `src/pages/RiskQueuePage.tsx`

Use this screen to demonstrate ranking:

1. Open **Priority Risk Queue** in the sidebar.
2. Search by patient name, patient code, or condition.

3. Choose a risk tier.
4. Filter intervention status.
5. Filter due dates such as today or next seven days.
6. Change sort field/order.
7. Change pages.
8. Click a patient to open details.
9. Use **Why?** to open factor explanations.
10. Use **What should I do?** to open action recommendations.
11. Use the intervention/contact buttons.
12. Export the visible risk population through CSV.

The queue is the best screen for the “Rank” part of the pitch.

Patient Records & Directory

File: `src/pages/PatientsPage.tsx`

This page supports:

- Search.
- Risk filtering.
- Pagination.
- Opening a patient.
- Calling a visible phone number through the browser’s `tel:` link.
- Admin-visible enrollment workflow.
- Address selection with the map picker.
- Edit patient modal.
- Analyzer workflow.
- Contact workflow.

The backend currently authorizes these routes with JWT authentication but does not consistently enforce the frontend’s admin-only claims. Explain that distinction if a judge asks about security.

Patient Details

File: `src/pages/PatientDetailsPage.tsx`

The page combines:

- Patient demographics and condition.
- Risk score and risk level.
- Risk factor breakdown.
- Appointment timeline.
- Existing interventions.
- Action recommendation.
- What-If simulator.
- Edit action.
- Clinical analyzer action.
- Contact action.
- Print/save report action.

This is the best screen for explaining one patient from raw inputs to an action plan.

Clinical Risk Analyzer

File: `src/pages/PredictionPage.tsx`

This page lets a user adjust risk-related inputs and see a calculated analysis. The UI calls `/api/analyzer/process` for server analysis. The code also recalculates the preview as values change.

The analyzer returns `AnalyzerFindings`, including:

- Patient identity.
- Risk score and tier.
- Evidence coverage.
- Primary drivers.
- Follow-up barriers.
- Clinical hazards.

- Recommended actions.
- Suggested intervention.
- A message draft.

The analyzer does not use Gemini. It uses the deterministic scoring engine and server-generated operational text.

Outreach and Dispatch

File: `src/pages/InterventionsPage.tsx`

This page lists intervention records and supports status updates. The main creation/contact experience is in `ContactPatientModal` and `InterventionModal`.

Typical status values include:

- `Pending`.
- `Contacted`.
- `Confirmed`.
- `Rescheduled`.
- `Completed`.
- `Escalated`.
- `Unable to Reach`.

Analytics

File: `src/pages/AnalyticsPage.tsx`

This page displays charts based on backend trend data and presentation-oriented KPI cards. Some headline business metrics are hardcoded for the demo. They are not a measured ROI calculation.

Use it to show:

- Risk trend visualization.
- Attendance trend visualization.
- Risk factor frequency.

- Intervention success chart.
- Operational impact narrative.

Settings

File: `src/pages/SettingsPage.tsx`

Settings has two tabs:

1. Scoring Engine

- Adjust maximum factor point caps.
- Save configuration.
- Reset to defaults.

1. Staff & User Registration

- Load staff list.
- Search staff.
- Register staff.
- Delete non-primary staff.

The sidebar exposes Settings only when the frontend user role is `ADMIN`. The server uses `requireRole('ADMIN')` specifically for updating scoring configuration.

Global modals and drawer

The global workflow components are mounted by `App.tsx` even though they are opened from different pages:

- Risk explanation.
- Recommended action.
- Intervention creation.
- Patient contact.
- Analyzer findings.

- Patient editing.
- Browser report/print.
- Gemini assistant.
- Admin login.

10. Risk Scoring Engine

Default weights

The default configuration in `DEFAULTSCORINGCONFIG` is:

FACTOR	MAXIMUM POINTS
Missed appointments	35
Distance	20
Attendance rate	20
Appointment frequency	10
Treatment duration	10
Age	5
Total	100

Actual formulas

1. Missed appointments

```
base = min(missedAppointments * 7, 35)
points = round((base / 35) * missedWeight)
```

With the default weight:

MISSED VISITS	POINTS
0	0
1	7
2	14
3	21
4	28
5 or more	35

2. Distance

```
base = min((distanceKm / 45) * 20, 20)
points = round((base / 20) * distanceWeight)
```

The score reaches the full 20 distance points at 45 km. The explanatory text treats 30 km as a significant barrier, but 30 km is not the full-point threshold.

3. Attendance

```
attendanceRate = round(attendedAppointments / totalAppointments * 100)
base = min(((100 - attendanceRate) / 50) * 20, 20)
points = round((base / 20) * attendanceWeight)
```

Attendance of 50% or below reaches the full attendance penalty.

4. Appointment cadence

```
base = min((appointmentFrequencyDays / 90) * 10, 10)
points = round((base / 10) * frequencyWeight)
```

The engine labels intervals over 60 days as infrequent, but the numeric score reaches the maximum at 90 days.

5. Treatment duration

```
base = min((treatmentDurationMonths / 24) * 10, 10)
points = round((base / 10) * durationWeight)
```

The explanatory language starts calling 12 months an extended journey; the numeric maximum occurs at 24 months.

6. Age

```
if age > 50:
    agePoints = min(((age - 50) / 50) * 5, 5)
else if age < 18:
    agePoints = min(((18 - age) / 18) * 5, 5)
else:
    agePoints = 0
```

The explanation text highlights age 65 and pediatric age below 12, but the numeric calculation begins increasing above 50 and below 18. This is an important code-versus-presentation discrepancy.

Total and tier

```
score = min(100, max(0, sum of six factor points))
```

Default tiers:

SCORE	TIER
0-29	LOW

SCORE	TIER
30-59	MEDIUM
60-79	HIGH
80-100	CRITICAL

The thresholds are stored in `ScoringConfiguration`. The scoring page can change weights, but always verify the resulting configuration because legacy threshold fields and nested threshold fields both exist in the type.

Scoring output

`calculatePatientRisk(...)` returns:

- `score` .
- `riskLevel` .
- `confidence` .
- `evidenceCoverage` .
- `predictionDate` .
- `modelVersion` .
- `reasons` .
- `protectiveFactors` .
- `recommendedActions` .
- `immediateAction` .
- `secondaryAction` .
- `alternativeAction` .
- `topFactors` .
- `naturalLanguageSummary` .
- `responsibleAiNote` .
- `inputSnapshot` .

Important interpretation rule

The score is an operational prioritization score, not a medical diagnosis and not a probability of readmission. Do not call `83` an “83% chance” of anything.

11. Patient Data Flow

Creating a patient

UI path:

Patients -> + Enroll New Patient -> fill form -> optional map picker -> save

Frontend:

- `PatientsPage.tsx` .
- `MapAddressPicker.tsx` .
- `api.createPatient(...)` .

Request:

POST /api/patients

Backend:

1. Read request body.
2. Calculate totals and attendance values.
3. Validate missed appointments are not greater than total.
4. Build an id and patient code.
5. Calculate current risk.
6. Save through `dbCreatePatient(...)` .
7. Log patient enrollment.

8. Return the new patient.

Database:

- Inserts into `patients` .
- May insert a prediction record depending on repository logic.
- Logs to `audit_logs` .

Reading a patient list

UI sends filters to `GET /api/patients` . The repository supports search, risk level, intervention status, due filter, sort, page, and limit.

The backend returns:

```
{
  "success": true,
  "data": [],
  "pagination": {
    "total": 1000,
    "page": 1,
    "limit": 15,
    "totalPages": 67
  }
}
```

Updating a patient

UI path:

Patients or Patient Details -> Edit -> save

Request:

PUT /api/patients/:id

The server updates fields, recalculates risk, writes the patient, and records an audit action.

Appointment history

Appointments are stored in the `appointments` table and returned as part of patient detail. The UI visualizes them through `AppointmentTimeline.tsx`.

The appointment data includes:

- Date.
- Department.
- Doctor.
- Status.
- Notes.

Intervention history

Interventions are stored in `interventions`. A patient detail response includes the patient's intervention records. The Interventions page lists them across patients.

Notification history

Notifications are stored in `notifications`, but the current application does not expose a dedicated notification history endpoint. The result of a contact workflow is shown in the modal/toast and represented through the linked intervention and audit log.

12. Backend API Reference

All routes are defined in `server.ts` . The Express app is the API; there are no active Vercel function wrappers.

Public routes

METHOD	PATH	PURPOSE
GET	/api	Health/root response
GET	/api/health	Health check for Render
POST	/api/auth/login	Authenticate staff and return JWT
POST	/api/auth/register	Register a staff account

Authenticated routes

METHOD	PATH	PURPOSE
GET	/api/users	List staff users
DELETE	/api/users/:email	Delete a staff user except primary admin
GET	/api/patients	Search/filter/paginate patients
GET	/api/patients/:id	Patient detail, appointments, interventions, risk
POST	/api/patients	Create patient
PUT	/api/patients/:id	Update patient
POST	/api/analyzer/process	Generate deterministic analyzer findings

METHOD	PATH	PURPOSE
POST	/api/patients/:id/contact	Contact workflow and notification/intervention record
POST	/api/predictions/predict	Calculate prediction from raw inputs
GET	/api/predictions	Read prediction history
GET	/api/patients/:id/risk	Calculate/read patient risk
POST	/api/interventions	Create intervention
GET	/api/interventions	List interventions
PUT	/api/interventions/:id/status	Update intervention status
GET	/api/dashboard/summary	Dashboard KPI summary
GET	/api/dashboard/risk-distribution	Risk distribution data
GET	/api/dashboard/trends	Trend/chart data
GET	/api/settings/config	Read scoring configuration
POST	/api/demo/reset	Reset demo data
POST	/api/demo/generate	Generate a synthetic dataset
POST	/api/assistant/chat	Gemini or fallback operations response
GET	/api/audit-logs	Read audit logs
GET	/api/export/csv	Download up to 1,000 patients as CSV

Admin-protected route

METHOD	PATH	PROTECTION
PUT	/api/settings/config	JWT plus <code>requireRole('ADMIN')</code>

The current backend does not apply `requireRole('ADMIN')` to every action that the UI describes as admin-only. This is a known security limitation and should be stated honestly.

API request example

```
$body = @{ email = 'admin@caretrack.in'; password = 'password123' } | ConvertTo-Json
$login = Invoke-WebRequest `
  -Uri http://localhost:3000/api/auth/login `
  -Method Post `
  -ContentType 'application/json' `
  -Body $body
$token = ($login.Content | ConvertFrom-Json).token

Invoke-WebRequest `
  -Uri 'http://localhost:3000/api/patients?limit=1' `
  -Headers @{ Authorization = "Bearer $token" }
```

Do not print or share the token outside a local test.

13. Database Architecture

Connection order

`src/db/db.ts` uses this order:

1. `DATABASE_URL`.
2. A direct local PostgreSQL connection on port 5432.
3. Embedded PostgreSQL on port 5433 for local development.
4. In-memory Maps/arrays if the allowed database options fail outside the production requirement.

Render should use step 1.

Tables

users

Stores staff identity, email, role, department, employee id, phone, password hash, and creation time.

patients

Stores patient identity, demographics, address, coordinates, distance, condition, treatment, appointment totals, attendance, risk JSON, latest intervention JSON, and timestamps.

appointments

Stores appointment date, department, doctor, status, notes, and patient id.

predictions

Stores score, tier, evidence, model version, reasons, protective factors, recommended actions, snapshots, and natural-language fields when a prediction is persisted.

interventions

Stores patient, prediction, staff, intervention type, status, notes, outcome, confirmation, creation time, and completion time.

notifications

Stores channel, destination, message content, status, provider, demo/live flag, and timestamp.

audit_logs

Stores timestamp, staff name, staff role, action, details, and optional patient code.

scoring_configs

Stores a JSON scoring configuration and update timestamp.

Startup and seeding

On first database initialization:

1. Tables and indexes are created.
2. The patient count is checked.
3. If there are no patients, the application seeds 1,000 records.
4. Seed users, scoring configuration, appointments, interventions, predictions, and audit logs are inserted.

The seed data includes ten canonical Indian demonstration patients plus generated patients.

Persistence warning

The in-memory store is useful for a local fallback/demo but is not durable across process restarts or multiple Render instances. A production Render deployment should connect successfully to the managed PostgreSQL database.

Embedded PostgreSQL warning

Embedded PostgreSQL is a local-development convenience. It writes under `.pgdata`, which is ignored by git. It is not a replacement for Render's managed database.

14. Gemini Clinical Operations Assistant

Where it lives

Frontend:

- `src/components/AiAssistantDrawer.tsx` .
- `src/services/api.ts` , method `askAssistant(...)` .

Backend:

- `server.ts` , `getGeminiClient()` .
- `POST /api/assistant/chat` .

Dependency:

```
"@google/genai": "^2.4.0"
```

Current model:

```
gemini-3.6-flash
```

What Gemini receives

The backend builds a prompt containing:

- Total patient count.
- High/critical patient count.
- Count of high-risk patients beyond 30 km.
- Optional context for a requested patient id.
- The user's question.

The instructions say to answer operational follow-up and prioritization questions only, and not diagnose or prescribe.

Example questions

The drawer suggests questions such as:

Why is patient P-1042 high risk?
How many high risk patients live beyond 30 km?

The backend detects patient codes and can produce patient-specific context.

Fallback mode

If there is no usable `GEMINI_API_KEY`, or Gemini returns an error, the backend returns deterministic responses based on keywords such as:

- `distance` .
- `far` .
- `km` .
- `high risk` .
- `queue` .
- `how many` .
- A patient code.

This means the demo can still show an assistant response without live Gemini. Be clear that fallback text is not generated by Gemini.

What is not implemented

The current assistant is not:

- A diagnostic assistant.

- A medication assistant.
- A medical decision maker.
- A guaranteed PHI-redacted service.
- An audited clinical language model gateway.

The code includes operational guardrails in the prompt, but production privacy and governance controls would require additional work.

Demo steps

1. Log in.
2. Click the AI assistant button in the navbar.
3. Ask `Why is patient P-1042 high risk?` .
4. Point out the operational explanation.
5. Explain that the score itself comes from the deterministic scoring engine.
6. If no Gemini key is configured, describe the visible response as deterministic demo fallback.

15. GIS and Map Features

Map component

File: `src/components/MapAddressPicker.tsx`

It uses:

- Leaflet.
- OpenStreetMap tiles.
- OpenStreetMap Nominatim search.
- Browser geolocation when permission is granted.
- Local Indian city and locality presets.

- A fixed reference hospital coordinate.
- Haversine distance calculation.

Reference coordinates:

Latitude: 28.5672
Longitude: 77.2100

How to access it

1. Log in as the admin demo user.
2. Open **Patient Records & Directory**.
3. Click + **Enroll New Patient**.
4. Open the address/map picker.
5. Search for a place, choose a preset, click the map, drag the marker, or use browser location.
6. Save the selected address and distance into the patient form.

How distance is calculated

The map component uses the Haversine formula to calculate rounded straight-line distance from the fixed hospital coordinate to the selected location.

It does not calculate driving time or a road route.

Presentation-guide discrepancy

The pitch guide describes radius friction rings at 10 km, 25 km, and 40+ km. Those rings are not implemented in the current `MapAddressPicker` component. The current map shows markers, tiles, selected location, address/search results, and a computed distance.

Seed patient distance values are stored in the data and are not always recomputed from their coordinates.

16. Communication and Twilio

Contact channels

The UI offers:

- Phone call.
- SMS.
- WhatsApp.

The backend sends all channels through `sendNotification(...)` , but provider behavior differs.

Actual provider behavior

CHANNEL	CURRENT BEHAVIOR
SMS with complete Twilio credentials	Attempts live Twilio Messages API call
SMS without credentials	Demo/simulated mode
SMS with Twilio error	Demo fallback
Phone call	Always simulated in current code
WhatsApp	Always simulated in current code

The code does not integrate Twilio Voice or WhatsApp APIs.

What is recorded?

Each notification receives:

- Notification id.
- Patient id.
- Channel.
- Destination.
- Message content.
- Status.
- Provider label.
- Demo/live flag.
- Timestamp.

The associated intervention and audit record are also created by the contact workflow.

Demo steps

1. Open a patient from the Risk Queue or Patient Directory.
2. Choose **Contact Patient**.
3. Select SMS or phone.
4. Review the generated message.
5. Submit.
6. Point out the demo-mode or live-mode result.
7. Open the intervention page to show the saved operational status.

Do not say that a real SMS was sent unless valid Twilio credentials were configured and the response says live Twilio succeeded.

17. Audit Logging and CSV Export

Audit events

The code logs many write and workflow actions, including:

- User login.
- Staff registration.
- Staff deletion.
- Patient enrollment.
- Patient updates.
- Analyzer processing.
- Patient contact.
- Intervention creation.
- Intervention status update.
- Scoring configuration changes.
- Demo reset.

Audit screen/access

The frontend calls:

```
GET /api/audit-logs
```

The API returns recent audit records. Some UI areas also show audit-related information through patient/intervention context.

What is not logged?

The current code does not log every read. Dashboard views, assistant questions, CSV exports, and most list reads are not automatically audit events.

The audit table is not immutable in a database-enforced sense. Demo reset deletes and recreates data, including audit logs. Do not describe it as tamper-proof or immutable without additional controls.

CSV export

The Risk Queue’s export action calls:

```
GET /api/export/csv
```

The export includes up to 1,000 patients with:

- Rank.
- Patient id.
- Name.
- Age.
- Risk score.
- Risk level.
- Missed visits.
- Distance.
- Attendance rate.
- Next follow-up.
- Immediate action.
- Intervention status.

The endpoint is authenticated.

18. Presentation Claims Compared with Code

This section should be read before presenting the project to judges.

PRESENTATION CLAIM	ACTUAL STATUS
Predict -> Explain -> Rank -> Intervene	Implemented as the main product workflow
Deterministic six-factor scoring	Implemented in <code>scoringEngine.ts</code>

PRESENTATION CLAIM	ACTUAL STATUS
Priya Patel / <code>P-1042</code> at 83	Implemented in canonical seed data with current defaults
Telehealth reduces Priya from 83 to 64	Implemented as a local hypothetical simulator scenario
Gemini assistant	Implemented with optional Gemini and deterministic fallback
Current Gemini model <code>gemini-3.6-flash</code>	Implemented in <code>server.ts</code>
Leaflet and OpenStreetMap map	Implemented in <code>MapAddressPicker.tsx</code>
Nominatim address search	Implemented in map picker
Radius rings	Not implemented
Live telehealth scheduling/video	Not implemented; represented as a hypothetical or intervention label
Live Twilio SMS	Implemented only when credentials exist; otherwise demo mode
Live phone calls and WhatsApp	Not implemented; simulated
PostgreSQL persistence	Implemented through <code>pg</code> and Render <code>DATABASE_URL</code>
PostgreSQL 18 everywhere	Not accurate; Render version is managed, Docker Compose uses PostgreSQL 16, local embedded dependency is PostgreSQL 18 beta
Vercel serverless deployment	Outdated; Vercel files/wrappers were removed and Render is current
Docker deployment	Implemented as a separate container path
Every action is RBAC protected	Not accurate; only some server routes enforce role-specific access

PRESENTATION CLAIM	ACTUAL STATUS
Immutable audit trail	Overstated; audit rows can be deleted by demo reset and there is no immutability mechanism
HIPAA/DPDPA/ABDM compliance	Not proven by code; no full compliance controls are implemented
ROI and no-show reduction metrics	Demo/presentation metrics; not calculated from validated outcomes
100% PHI removal from Gemini requests	Not guaranteed; patient context can include name, code, age, distance, attendance, and reasons
EMR/FHIR connectors	Roadmap / not currently implemented
IVR voice bot	Roadmap / not currently implemented
ASHA mobile app	Roadmap / not currently implemented
Java backend architecture	Presentation/demo source only; the active backend is TypeScript/Express

Claims to avoid

Do not say:

- “This is a clinically validated prediction model.”
- “An 83 score means an 83% probability.”
- “The system is HIPAA compliant because it uses JWT.”
- “Every patient action is fully RBAC protected.”
- “Every message is sent live through Twilio.”
- “Telehealth sessions are created by the app.”
- “The map contains implemented radius rings.”
- “The ROI numbers were measured by this application.”
- “The current deployment is Vercel.”

Use:

- “Explainable operational risk score.”
- “Synthetic/demo data.”
- “Optional Gemini integration with deterministic fallback.”
- “Twilio SMS when configured; simulated channels otherwise.”
- “Render web service with managed PostgreSQL.”

19. Three-Minute Demo Script

This sequence follows the supplied presentation guide while matching the current UI.

0:00-0:30: Login

Action: Open the app and log in as `admin@caretrack.in` with `password123` , or use the quick admin login control.

Say:

We begin with a role-aware hospital operations workspace. The user signs in through the Express API, receives a JWT, and the browser loads the authenticated dashboard. This is a demo account and not a production credential.

0:30-1:00: Dashboard

Action: Point to the KPI cards, top-risk table, chart area, and Priority Queue button.

Say:

The dashboard summarizes the outpatient cohort and directs staff toward the highest-priority follow-ups. These values are based on the seeded synthetic dataset and server calculations, with some presentation fallback values in the UI.

1:00-1:35: Risk Queue

Action: Open **Priority Risk Queue**. Filter to high risk or next seven days. Search for **P-1042** if needed.

Say:

The queue turns the score into an operational order of work. Staff can search, filter by tier and due window, sort by risk, and open a patient for explanation and action.

1:35-2:00: Priya Patel details

Action: Open Priya Patel, then show the score, risk factors, appointment timeline, and recommended action.

Say:

Priya Patel is the canonical demo patient. Her record contains five missed visits, 42 kilometers of travel distance, 58% attendance, a 60-day cadence, and 12 months of treatment. With the default six-factor formula, those values produce 83, or CRITICAL. This is an operational score, not a medical diagnosis or probability.

2:00-2:20: What-If simulator

Action: Click **Simulate Telehealth** and point to the score change.

Say:

The simulator is hypothetical and runs locally in the browser. It changes effective travel distance to zero and recalculates the score. Priya's displayed scenario moves from 83 to 64, a 19-point reduction. The application has not booked a video visit; it has shown the potential effect of removing travel friction.

2:20-2:40: Contact workflow

Action: Open **Contact Patient**, choose SMS or phone, review the draft, submit.

Say:

The coordinator can now record an outreach attempt. With Twilio credentials, SMS can call the Twilio Messages API. Without them, the application records a transparent demo-mode notification. Phone and WhatsApp are simulated in the current code.

2:40-3:00: Gemini and audit

Action: Open the AI assistant and ask `Why is patient P-1042 high risk?`. Then show the intervention or audit area.

Say:

The assistant is constrained to operational questions and uses the current `gemini-3.6-flash` model when a Gemini key is configured. If Gemini is unavailable, deterministic fallback text keeps the demo usable. The contact and intervention workflow records operational history in PostgreSQL.

20. Longer Presentation Scripts

Slide 1: Title and vision

Say:

CareTrack AI is an explainable outpatient follow-up operations platform. Its workflow is Predict, Explain, Rank, Intervene, and then record the outcome. It is built as a React and Express application with PostgreSQL persistence and optional external services.

Do not display “Vercel Ready” as a current deployment badge. The current deployment target is Render.

Slide 2: Problem

Say:

The application addresses the operational problem of missed follow-up appointments and the friction behind them. The repository models this with missed visits, distance, attendance, cadence, duration, age, transport, and language fields. External no-show and financial statistics belong to the presentation context, not to this application’s measured output.

Slide 3: Solution

Say:

The solution connects a transparent score to a queue and a workflow. The user can move from risk identification to factor explanation, hypothetical simulation, contact, intervention status, and audit

record without leaving the main workspace.

Slide 4: Scoring engine

Say:

The engine has six bounded factors with default caps adding to 100. The code uses explicit formulas. The current implementation's numeric thresholds should be explained accurately: distance reaches its full numeric contribution at 45 kilometers, cadence at 90 days, duration at 24 months, and age scoring begins above 50 or below 18 even though the explanatory text emphasizes 65 and 12.

Slide 5: Hero patient

Say:

Priya Patel is a synthetic canonical patient. Her 83 score is reproducible from the current seed data and default scoring configuration. The telehealth scenario is a hypothetical distance-zero simulation, not a live telehealth booking.

Slide 6: GIS

Say:

The map picker uses Leaflet tiles, Nominatim search, local Indian place presets, browser location, draggable markers, and Haversine distance from a fixed reference hospital. Radius rings and route-driving distance are not currently implemented.

Slide 7: Gemini

Say:

Gemini is an optional operations assistant. It can explain risk drivers and summarize operational counts. Its prompt prohibits diagnosis and prescriptions. The deterministic engine remains responsible for the actual score. When no key is configured or Gemini fails, the backend uses keyword-based fallback responses.

Slide 8: Architecture

Say:

The active architecture is one Node process: Vite builds the client, esbuild bundles the Express server, and `npm start` runs the compiled server. Render provides the web service and managed PostgreSQL connection. Docker Compose is an alternative local/container path, not the Render runtime.

Slide 9: Roles and governance

Say:

The UI exposes five roles and uses JWT authentication. Settings visibility is restricted in the frontend, and scoring configuration updates are admin-protected on the backend. Other write routes need additional server-side role hardening before claiming complete enterprise RBAC.

Slide 10: Impact

Say:

The application shows operational impact-style charts and demo KPIs. These are not a validated ROI model or a measured hospital outcome study. A real impact evaluation would require production baseline data, controlled intervention definitions, and outcome measurement.

Slide 11: Differentiation

Say:

The strongest defensible differentiator is the combination of a transparent factor score, a ranked work queue, a hypothetical simulator, and an action record. Avoid saying that the current system is a complete replacement for an EHR or a production clinical AI platform.

Slide 12: Roadmap

The presentation guide's FHIR/EMR connectors, IVR voice bot, and ASHA mobile application are roadmap items. They are not implemented in this repository.

21. Production Deployment

Render Blueprint

The repository contains `render.yaml`.

It declares:

```
services:
  - type: web
    runtime: node
    buildCommand: npm ci && npm run build
    startCommand: npm start
    healthCheckPath: /api/health

databases:
  - name: caretrack-db
```

Render should create the database and inject its connection string into the web service's `DATABASE_URL` variable.

Deployment steps

1. Push the repository to GitHub.
2. Open Render.
3. Choose **New** and **Blueprint**.
4. Connect the repository and select the `main` branch.
5. Let Render detect `render.yaml`.
6. Review the web service and PostgreSQL database resources.
7. Add optional `GEMINI_API_KEY` if live Gemini is required.
8. Add Twilio variables only if live SMS is required.
9. Deploy.
10. Check the Render logs for database connection and seed messages.
11. Open the generated service URL.

12. Check `/api/health` .
13. Log in with a demo account only for demonstration.

Expected production startup

A healthy startup should:

1. Start the Node process.
2. Read `DATABASE_URL` .
3. Connect to PostgreSQL.
4. Create missing tables/indexes.
5. Seed the database if it has no patients.
6. Start listening on Render's `PORT` .
7. Pass the `/api/health` check.

Production checks

```
npm ci
npm run lint
npm run build
npm start
```

On Render, do not rely on `.pgdata` . The durable database should be the managed PostgreSQL resource.

Docker deployment

The Docker path is separate:

```
docker build -t caretrack-ai .
docker run --env-file .env -p 3000:3000 caretrack-ai
```

For Compose:

```
docker compose up --build
```

The Compose file uses a local PostgreSQL service and has its own credentials. Keep those values out of public production environments.

22. Safe Modification Guide

Before changing anything

1. Read the relevant component and its API method.
2. Search for all usages of the symbol.
3. Check shared types.
4. Check the corresponding Express route.
5. Check the database repository method.
6. Run the narrowest useful test or type check before editing.
7. Inspect `git status` so existing work is not overwritten.

Common change locations

DESIRED CHANGE	START HERE	ALSO INSPECT
Change score formula	<code>src/services/scoringEngine.ts</code>	<code>RiskSimulator</code> , server prediction paths, Settings
Add a risk factor	<code>scoringEngine.ts</code>	<code>types.ts</code> , UI factor displays, seed/demo data
Change patient fields	<code>types.ts</code> , <code>db.ts</code>	SQL schema, mapper, forms, API route

DESIRED CHANGE	START HERE	ALSO INSPECT
Add an API endpoint	<code>server.ts</code>	<code>src/services/api.ts</code> , calling page, auth, DB method
Change dashboard KPI	<code>DashboardPage.tsx</code> , <code>db.ts</code>	<code>AnalyticsPage</code> , API response type
Change Gemini model/prompt	<code>server.ts</code>	<code>package.json</code> , env docs, assistant drawer
Change Twilio behavior	<code>notificationService.ts</code>	<code>ContactPatientModal</code> , notification DB method
Change map behavior	<code>MapAddressPicker.tsx</code>	<code>PatientsPage</code> , seed city data, external service assumptions
Add navigation page	<code>App.tsx</code> , <code>Sidebar.tsx</code> , <code>AppContext.tsx</code>	page component and access rules
Change staff workflow	<code>SettingsPage.tsx</code> , <code>AuthContext.tsx</code>	auth endpoints, password handling, role checks
Change deployment	<code>render.yaml</code> , <code>Dockerfile</code> , <code>package.json</code>	environment docs and health endpoint

Do not make these assumptions

- A UI-hidden action is automatically secure. Verify backend authorization.
- A displayed KPI is necessarily calculated from live data.
- A notification with “SMS” necessarily went through Twilio.
- A telehealth label means a video session exists.
- A risk score is a probability.
- A seeded patient date is fixed; several dates are generated relative to runtime date.
- A database fallback is durable.

- A prompt instruction alone provides compliance.

Safe feature change pattern

For a backend-backed feature:

Shared type

- > database schema/repository
- > Express route
- > frontend api.ts method
- > context/page/component
- > focused test or endpoint check
- > full type/build check

Keep each change narrow and preserve existing response shapes unless a deliberate API migration is planned.

23. Troubleshooting

`npm start` says package.json is missing

Make sure the shell is in the project directory:

```
Set-Location C:\d_backup\PROJECTS\CareTrackAi\NextEra---ASA-Trinity  
npm start
```

Or use an explicit prefix:

```
npm.cmd --prefix "C:\d_backup\PROJECTS\CareTrackAi\NextEra---ASA-Trinity" start
```

Port 3000 is already in use

Find the process:

```
Get-NetTCPConnection -LocalPort 3000
```

Stop only the process you recognize, or set another local `PORT` in `.env`.

Embedded PostgreSQL startup is slow

The first run downloads/initializes a local database cluster. Wait for the server to print that PostgreSQL is ready and that the application is listening.

`.pgdata` is not empty / database restarts strangely

This is a local embedded database issue. Stop the old Node/PostgreSQL process cleanly. Do not delete `.pgdata` if you need its local data. If it is disposable demo data, stop all related processes and remove `.pgdata`, then restart.

Production says `DATABASE_URL` must be configured

Render did not inject the database connection string. Confirm:

1. The PostgreSQL resource exists.
2. The web service is linked to it in Render.
3. The environment variable is named exactly `DATABASE_URL`.
4. The service was redeployed after configuration.

PostgreSQL connection fails

Check:

- Connection string format.
- SSL requirements.
- Database availability.
- Render internal/external connection choice.
- Pool limit.
- Service logs.

The application uses SSL for non-local `DATABASE_URL` connections with `rejectUnauthorized: false`. Review this policy before a regulated production deployment.

Login fails

Check:

- API is running.
- Database initialized and seed users exist.
- Email is one of the seed accounts.
- Password is `password123` for demo accounts.
- Browser local storage is not holding a stale token.

Clear `caretracktoken` and `caretrack user` in browser storage when testing a fresh session.

Dashboard is blank or shows fallback values

Open browser developer tools and inspect `/api/dashboard/summary`, `/api/settings/config`, and `/api/dashboard/trends`. Confirm:

- The JWT exists.
- The API returns `success: true`.
- The database is initialized.

- The response shape matches `src/services/api.ts`.

Gemini assistant shows fallback text

That is expected when:

- `GEMINI_API_KEY` is absent.
- The key is a placeholder.
- Gemini rejects the request.
- The network request fails.

Check server logs for `[Gemini Assistant Error]`. The current request uses `gemini-3.6-flash`.

Twilio says demo mode

Demo mode is expected unless all three variables are configured:

- `TWILIO_ACCOUNT_SID`.
- `TWILIO_AUTH_TOKEN`.
- `TWILIO_PHONE_NUMBER`.

Only SMS attempts the live Twilio API. Phone and WhatsApp remain simulated.

Map search does not return results

Map search depends on Nominatim network access. Try:

- A local city preset.
- A known Indian locality.
- A map click.
- Browser GPS permission.

Remember that the map calculates straight-line distance, not driving distance.

TypeScript errors after a change

Run:

```
npm run lint
```

Read the first error and follow its import/type path. Avoid changing unrelated files to silence errors.

Render health check fails

Test locally:

```
http://localhost:3000/api/health
```

Then verify:

- Render is using `npm run build`.
- Render is using `npm start`.
- The service listens on `0.0.0.0`.
- The database initialization completes.
- Render health check path is `/api/health`.

24. Judge Q&A

Why deterministic scoring instead of deep learning?

Accurate answer:

This prototype prioritizes transparency and operational explainability. Each score is built from six explicit bounded factors, so staff can inspect why a patient moved up the queue. It is not presented as a validated clinical prediction probability. A future production model could be evaluated against outcomes while retaining explanation and governance requirements.

Does the score predict readmission?

No. It scores follow-up attendance friction and prioritization. The repository does not implement a readmission model or validate a readmission relationship.

What happens if PostgreSQL is unavailable?

The code contains local fallback behavior, including embedded PostgreSQL and an in-memory dataset outside the production `DATABASE_URL` requirement. In-memory data is not durable and should not be described as a production database outage solution without additional reliability design.

Is the data real?

No. The repository seeds synthetic/demo patients and accounts. Priya Patel is a canonical synthetic demonstration patient.

Is the system HIPAA or DPDPA compliant?

Do not claim that based on this code. JWT authentication and audit records are useful building blocks, but the repository does not demonstrate complete encryption, consent, tenant isolation, data retention, PHI controls, secure secret rotation, or formal compliance certification.

Is Gemini making the risk decision?

No. The deterministic scoring engine calculates the score. Gemini is an optional operational assistant that summarizes information and answers constrained questions. A deterministic fallback exists.

What if Gemini is unavailable?

The server catches Gemini errors and returns keyword-based deterministic operational responses. This keeps the demo functional but is not equivalent to Gemini quality.

Is telehealth integrated?

No. The application can simulate a distance-zero scenario and record a teleconsultation-related intervention or message. It does not create a video meeting, book a telehealth appointment, or integrate with an EHR scheduler.

Is Twilio fully integrated?

Only partially. SMS can use the Twilio Messages API when credentials exist. Phone calls and WhatsApp are simulated. Errors fall back to demo recording.

Is the map a routing system?

No. It is an address/location picker using Leaflet, OpenStreetMap, Nominatim, presets, GPS, and straight-line Haversine distance. It has no road routing or radius rings.

What does Render deploy?

Render runs the compiled Express server, which serves the React build and the API. The same service uses the Render PostgreSQL connection from `DATABASE_URL`.

What is the strongest demo point?

The strongest defensible flow is:

```
Priya Patel -> explain 83 -> simulate telehealth -> show 64 -> record contact -> show  
intervention/audit
```

It demonstrates a complete chain without claiming that the simulator itself performed a real clinical intervention.

What is still roadmap?

The presentation roadmap items are not currently implemented:

- EMR/FHIR connectors.
- Epic/Cerner integrations.
- ABDM integration.
- Multilingual IVR voice bot.
- ASHA/community-worker mobile app.
- Real video/telehealth scheduling.
- Full production compliance controls.

25. Final Beginner Checklist

Before a demo

- ☐ Run `npm install` or `npm ci`.
- ☐ Confirm the server starts.
- ☐ Confirm `/api/health` returns healthy.
- ☐ Confirm login works.
- ☐ Confirm the dashboard loads.
- ☐ Search for `P-1042`.
- ☐ Confirm Priya's score is 83 with default configuration.
- ☐ Confirm the simulator shows 64 when telehealth is active.
- ☐ Decide whether the demo uses real Gemini or fallback mode.
- ☐ Decide whether the demo uses live SMS or Twilio demo mode.
- ☐ Do not claim unsupported compliance, ROI, Vercel, radius rings, or live telehealth.

Before changing code

- ☐ Check `git status`.
- ☐ Read the owning component/service.
- ☐ Search all symbol usages.
- ☐ Check the API contract.
- ☐ Check the database method and schema.
- ☐ Keep secrets out of source and `VITE_*` variables.
- ☐ Run `npm run lint`.
- ☐ Run `npm run build`.
- ☐ Test the affected workflow.
- ☐ Review `git diff`.

Before deploying Render

- ☐ `render.yaml` is present.
- ☐ PostgreSQL resource is created.
- ☐ `DATABASE_URL` is linked.

- [] `JWT_SECRET` is generated/configured.
- [] Optional Gemini/Twilio variables are added only when needed.
- [] Build command is `npm ci && npm run build`.
- [] Start command is `npm start`.
- [] Health check is `/api/health`.
- [] Logs show successful PostgreSQL connection.
- [] The deployed URL opens the React app.
- [] Login and one authenticated API request succeed.

One-sentence project summary

CareTrack AI is a synthetic-data, explainable outpatient follow-up operations prototype that uses a deterministic six-factor score to prioritize patients, explain barriers, simulate interventions, and record staff outreach through a React/Express/PostgreSQL application deployed on Render.

Generated from the repository's current CARETRACK_AI_COMPLETE_WALKTHROUGH.md. The Markdown remains the source of truth.